

Отчет по домашнему заданию

```
Код: import telebot
from telebot import types
import json
import datetime
import os

BOT_TOKEN = "8339778675:AAGZ0zxWDoXX7PoFSf-jCgzTT3JPuaElau4"
bot = telebot.TeleBot(BOT_TOKEN)

DATA_FILE = "planner_data.json"

def load_data():
    if os.path.exists(DATA_FILE):
        with open(DATA_FILE, 'r', encoding='utf-8') as f:
            return json.load(f)
    return {}

def save_data(data):
    with open(DATA_FILE, 'w', encoding='utf-8') as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def get_user_data(user_id):
    data = load_data()
    if str(user_id) not in data:
        data[str(user_id)] = {"tasks": [], "notes": []}
    save_data(data)
    return data[str(user_id)]

@bot.message_handler(commands=['start'])
def start(message):
    markup = types.ReplyKeyboardMarkup(resize_keyboard=True)
    btn1 = types.KeyboardButton("📝 Добавить задачу")
    btn2 = types.KeyboardButton("📁 Мои задачи")
    btn3 = types.KeyboardButton("🗑 Удалить задачу")
    btn4 = types.KeyboardButton("📌 Заметки")
    markup.add(btn1, btn2, btn3, btn4)

    bot.send_message(message.chat.id,
```

```

        "\ud83d\udccb *Добро пожаловать в PlannerBot!\n\n"
        "Ваш личный помощник для планирования задач.\n"
        "Выберите действие:",
        parse_mode='Markdown',
        reply_markup=markup)

@bot.message_handler(func=lambda message: message.text == "\ud83d\udccb Добавить задачу")
def add_task_start(message):
    msg = bot.send_message(message.chat.id, "\ud83d\udccb Введите название задачи:")
    bot.register_next_step_handler(msg, add_task_description)

def add_task_description(message):
    task_name = message.text
    if len(task_name) < 2:
        bot.send_message(message.chat.id, "\u274c Название слишком короткое")
        return

    user_data = get_user_data(message.from_user.id)
    task_id = len(user_data["tasks"]) + 1

    user_data["tasks"].append({
        "id": task_id,
        "name": task_name,
        "description": "",
        "date": datetime.datetime.now().strftime("%Y-%m-%d %H:%M"),
        "completed": False
    })

    data = load_data()
    data[str(message.from_user.id)] = user_data
    save_data(data)

    markup = types.InlineKeyboardMarkup()
    btn1 = types.InlineKeyboardButton("\ud83d\udccb Добавить описание", callback_data=f"desc_{task_id}")
    btn2 = types.InlineKeyboardButton("\u2705 Завершить", callback_data=f"complete_{task_id}")
    markup.add(btn1, btn2)

    bot.send_message(message.chat.id,
        f"\u2705 Задача добавлена!\n\n"
        f"\ud83d\udccb *{task_name}*\n"
        f"\ud83d\udccb {datetime.datetime.now().strftime('%d.%m.%Y %H:%M')}\n"

```

```
f"📝 Описание: нет",
parse_mode='Markdown',
reply_markup=markup)
```

```
@bot.message_handler(func=lambda message: message.text == "📋 Мои задачи")
```

```
def show_tasks(message):
```

```
    user_data = get_user_data(message.from_user.id)
```

```
    tasks = user_data["tasks"]
```

```
    if not tasks:
```

```
        bot.send_message(message.chat.id, "📋 Список задач пуст")
```

```
        return
```

```
    completed = [t for t in tasks if t["completed"]]
```

```
    active = [t for t in tasks if not t["completed"]]
```

```
    text = "📋 *Ваши задачи*\n\n"
```

```
    if active:
```

```
        text += "🕒 *Активные:*\n"
```

```
        for task in active[:10]:
```

```
            status = "✅" if task["completed"] else "⌚"
```

```
            text += f"{status} {task['id']}. {task['name']}\n"
```

```
            if task['description']:
```

```
                text += f"📝 {task['description'][:30]}...\n"
```

```
            text += f"📅 {task['date']}\n\n"
```

```
    if completed:
```

```
        text += "🕒 *Выполненные:*\n"
```

```
        for task in completed[:5]:
```

```
            text += f"✅ {task['id']}. {task['name']}\n"
```

```
    if len(tasks) > 15:
```

```
        text += f"\n📊 Всего задач: {len(tasks)}"
```

```
    bot.send_message(message.chat.id, text, parse_mode='Markdown')
```

```
@bot.message_handler(func=lambda message: message.text == "🗑 Удалить задачу")
```

```
def delete_task_start(message):
```

```
    user_data = get_user_data(message.from_user.id)
```

```
    tasks = user_data["tasks"]
```

```

if not tasks:
    bot.send_message(message.chat.id, "🗑 Нет задач для удаления")
    return

markup = types.InlineKeyboardMarkup()
for task in tasks[:10]:
    btn = types.InlineKeyboardButton(
        f"{task['id']}. {task['name'][:20]}",
        callback_data=f"delete_{task['id']}"
    )
    markup.add(btn)

bot.send_message(message.chat.id,
    "🗑 Выберите задачу для удаления:",
    reply_markup=markup)

@bot.message_handler(func=lambda message: message.text == "📝 Заметки")
def notes_menu(message):
    markup = types.InlineKeyboardMarkup()
    btn1 = types.InlineKeyboardButton("📝 Добавить заметку", callback_data="add_note")
    btn2 = types.InlineKeyboardButton("📖 Мои заметки", callback_data="show_notes")
    markup.add(btn1, btn2)

    bot.send_message(message.chat.id,
        "📖 *Управление заметками*",
        parse_mode='Markdown',
        reply_markup=markup)

@bot.callback_query_handler(func=lambda call: True)
def callback_handler(call):
    user_id = call.from_user.id
    user_data = get_user_data(user_id)

    if call.data.startswith("desc_"):
        task_id = int(call.data.split("_")[1])
        msg = bot.send_message(call.message.chat.id, "📝 Введите описание задачи:")
        bot.register_next_step_handler(msg, lambda m: add_description(m, task_id, call.message.message_id))

    elif call.data.startswith("complete_"):
        task_id = int(call.data.split("_")[1])

```

```

for task in user_data["tasks"]:
    if task["id"] == task_id:
        task["completed"] = True
        break

data = load_data()
data[str(user_id)] = user_data
save_data(data)

bot.answer_callback_query(call.id, "✅ Задача завершена!")
bot.edit_message_text("✅ Задача завершена!",
                      call.message.chat.id,
                      call.message.message_id)

elif call.data.startswith("delete_"):
    task_id = int(call.data.split("_")[1])
    user_data["tasks"] = [t for t in user_data["tasks"] if t["id"] != task_id]

    data = load_data()
    data[str(user_id)] = user_data
    save_data(data)

    bot.answer_callback_query(call.id, "🗑️ Задача удалена")
    bot.edit_message_text("🗑️ Задача удалена",
                          call.message.chat.id,
                          call.message.message_id)

elif call.data == "add_note":
    msg = bot.send_message(call.message.chat.id, "📝 Введите текст заметки:")
    bot.register_next_step_handler(msg, add_note)

elif call.data == "show_notes":
    notes = user_data.get("notes", [])
    if not notes:
        bot.send_message(call.message.chat.id, "📄 Нет заметок")
        return

    text = "📋 *Ваши заметки:*\\n\\n"
    for i, note in enumerate(notes[:10], 1):
        text += f"{i}. {note['text'][:50]}...\\n"
        text += f"📅 {note['date']}\\n\\n"

```

```

bot.send_message(call.message.chat.id, text, parse_mode='Markdown')

def add_description(message, task_id, original_msg_id):
    user_data = get_user_data(message.from_user.id)

    for task in user_data["tasks"]:
        if task["id"] == task_id:
            task["description"] = message.text
            break

    data = load_data()
    data[str(message.from_user.id)] = user_data
    save_data(data)

    bot.delete_message(message.chat.id, original_msg_id)
    bot.send_message(message.chat.id, "✅ Описание добавлено!")

def add_note(message):
    user_data = get_user_data(message.from_user.id)

    if "notes" not in user_data:
        user_data["notes"] = []

    note_id = len(user_data["notes"]) + 1
    user_data["notes"].append({
        "id": note_id,
        "text": message.text,
        "date": datetime.datetime.now().strftime("%Y-%m-%d %H:%M")
    })

    data = load_data()
    data[str(message.from_user.id)] = user_data
    save_data(data)

    bot.send_message(message.chat.id, "✅ Заметка сохранена!")

@bot.message_handler(commands=['stats'])
def stats(message):
    user_data = get_user_data(message.from_user.id)
    tasks = user_data["tasks"]

```

```
notes = user_data.get("notes", [])
```

```
completed = len([t for t in tasks if t["completed"]])
```

```
active = len([t for t in tasks if not t["completed"]])
```

```
text = f"""
```

```
 *Статистика*
```

```
 Всего задач: {len(tasks)}
```

```
 Выполнено: {completed}
```

```
 Активных: {active}
```

```
 Заметок: {len(notes)}
```

```
 Прогресс: {completed}/{len(tasks) if tasks else 1}
```

```
"""
```

```
bot.send_message(message.chat.id, text, parse_mode='Markdown')
```

```
@bot.message_handler(commands=['clear'])
```

```
def clear_all(message):
```

```
    markup = types.InlineKeyboardMarkup()
```

```
    btn1 = types.InlineKeyboardButton(" Удалить ВСЕ задачи", callback_data="clear_tasks")
```

```
    btn2 = types.InlineKeyboardButton(" Удалить ВСЕ заметки", callback_data="clear_notes")
```

```
    markup.add(btn1, btn2)
```

```
bot.send_message(message.chat.id,
```

```
    "⚠️ *Очистка данных*",
```

```
    parse_mode='Markdown',
```

```
    reply_markup=markup)
```

```
@bot.message_handler(func=lambda message: True)
```

```
def handle_text(message):
```

```
    if message.text.lower() in ['привет', 'hello', 'hi']:
```

```
        bot.send_message(message.chat.id, f" Привет, {message.from_user.first_name}!")
```

```
    elif 'спасибо' in message.text.lower():
```

```
        bot.send_message(message.chat.id, " Пожалуйста!")
```

```
    else:
```

```
        bot.send_message(message.chat.id,
```

```
            "Используйте кнопки меню или команды:\n"
```

```
            "/start - Главное меню\n"
```

```
            "/stats - Статистика\n"
```

```
            "/clear - Очистка данных")
```

```
print("🤖 PlannerBot запущен...")
```

```
bot.polling(none_stop=True)
```

Пример

